

AGEX Runtime Architecture

1. 문서 개요

1.1 문서 목적

본 문서는 AGEX Runtime의 구조, 책임, 실행 모델, 상태 관리, 자원 통제, 실패 처리, 복구 방식 및 운영 기준을 정의한다.

AGEX Runtime은 Agent, Workflow, Plugin, Model Task 및 기타 실행 단위가 실제로 동작하는 공통 실행 환경이다.

본 문서는 다음 설계의 기준으로 사용한다.

- Runtime 구성요소 정의
- 실행 요청 수신 및 검증
- 실행 단위 관리
- 실행 상태 관리
- Task Scheduling
- 자원 할당 및 제한
- Agent 실행
- Workflow Step 실행
- Plugin 실행
- Model 호출
- Sandbox 실행
- Timeout 및 Retry
- Checkpoint 및 복구
- 동시성 관리
- 장기 실행 처리
- 실행 중단 및 취소
- 실행 결과 저장
- 비용 측정

- Runtime 보안
- Runtime 관측성
- Runtime 확장 및 배포

1.2 적용 범위

본 문서는 다음 Runtime 영역을 포함한다.

- Runtime Control Plane
- Runtime Data Plane
- Execution Manager
- Execution Scheduler
- Task Dispatcher
- Task Executor
- Agent Runtime
- Workflow Runtime
- Plugin Runtime
- Model Execution Runtime
- Sandbox Runtime
- State Manager
- Checkpoint Manager
- Retry Manager
- Timeout Manager
- Recovery Manager
- Compensation Manager
- Resource Manager
- Queue Manager
- Runtime Policy Enforcement

- Runtime Observability
- Runtime Security Boundary

1.3 Runtime의 정의

AGEX Runtime은 검증된 Execution Request를 받아 실제 작업으로 변환하고, 실행 상태와 자원을 통제하며, 결과를 저장하고 반환하는 실행 계층이다.

Runtime은 단순한 함수 실행기나 Job Queue가 아니다.

Runtime은 다음을 통합적으로 수행한다.

- 실행 구성 확정
- 실행 환경 생성
- 권한과 정책 재검증
- Task 분해 및 배치
- Agent와 Workflow 실행
- Model과 Plugin 호출
- Knowledge와 Memory 연결
- 상태 지속성 유지
- 자원 사용 통제
- 실패 감지
- 재시도 및 복구
- 실행 중지 및 취소
- 비용 및 사용량 기록
- 로그, Metric, Trace 및 Audit 생성

2. Runtime Architecture 목표

AGEX Runtime은 다음 목표를 충족해야 한다.

2.1 실행 일관성

모든 Agent, Workflow 및 Task는 동일한 Runtime 규칙과 상태 모델을 따라야 한다.

2.2 지속 가능한 실행

장시간 실행, 외부 Callback 대기, 승인 대기 및 시스템 재시작 상황에서도 실행 상태가 유지되어야 한다.

2.3 정책 기반 통제

모든 실행은 권한, 정책, 비용, 시간, 자원 및 네트워크 제한 안에서 수행되어야 한다.

2.4 실패 격리

하나의 Execution, Task, Plugin 또는 Tenant에서 발생한 실패가 다른 실행이나 Platform 전체로 확산되지 않아야 한다.

2.5 복구 가능성

실행 중 장애가 발생해도 마지막 유효 Checkpoint부터 복구할 수 있어야 한다.

2.6 수평 확장

Runtime Worker와 Queue Consumer는 부하에 따라 독립적으로 확장할 수 있어야 한다.

2.7 비결정성 통제

Model 결과가 확률적으로 달라질 수 있더라도 실행 구성, 상태 변화 및 사용된 자원은 추적 가능해야 한다.

2.8 실행 격리

Tenant, Execution, Plugin 및 외부 코드의 실행 경계를 분리해야 한다.

2.9 관측 가능성

실행의 시작, 진행, 대기, 실패, 재시도, 중단 및 완료 상태를 실시간으로 추적할 수 있어야 한다.

2.10 비용 통제

Model, Plugin, Compute, Storage 및 Network 사용량을 실행 단위로 측정하고 제한할 수 있어야 한다.

3. Runtime 설계 원칙

AGEX Runtime은 다음 원칙을 따른다.

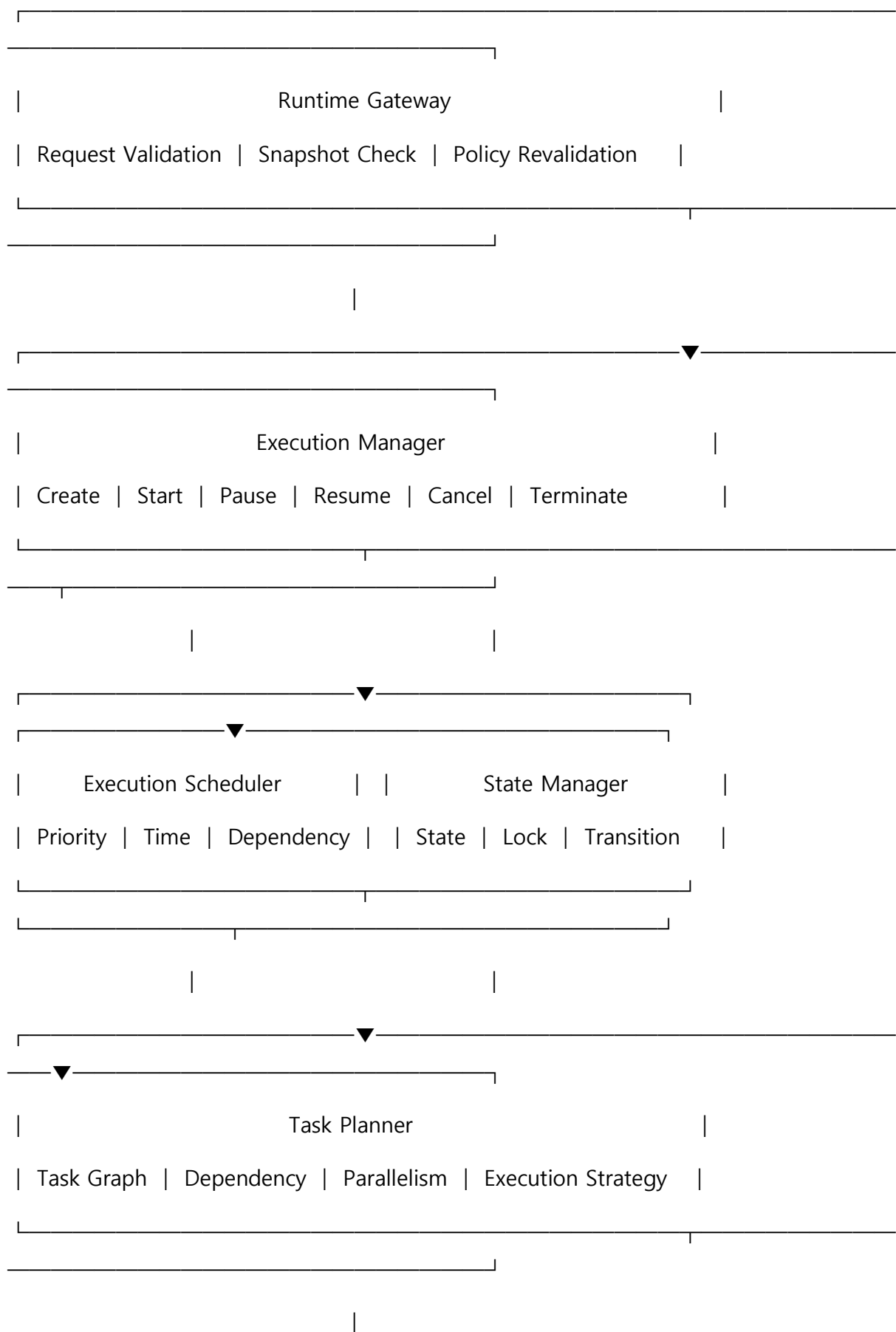
- Runtime-Centered Execution
- Durable Execution
- Explicit State Machine
- At-Least-Once Delivery
- Idempotent Task Execution
- Immutable Execution Snapshot
- Policy Before Action
- Least Privilege
- Tenant Isolation
- Resource Quota Enforcement
- Checkpoint-Based Recovery
- Failure Isolation
- Bounded Retry
- Bounded Execution Time
- Cancellation Support
- Horizontal Scalability
- Stateless Worker
- Externalized State
- Event-Driven Coordination
- Observable by Default
- Secure Sandbox
- Deterministic Control Logic
- Version-Pinned Execution

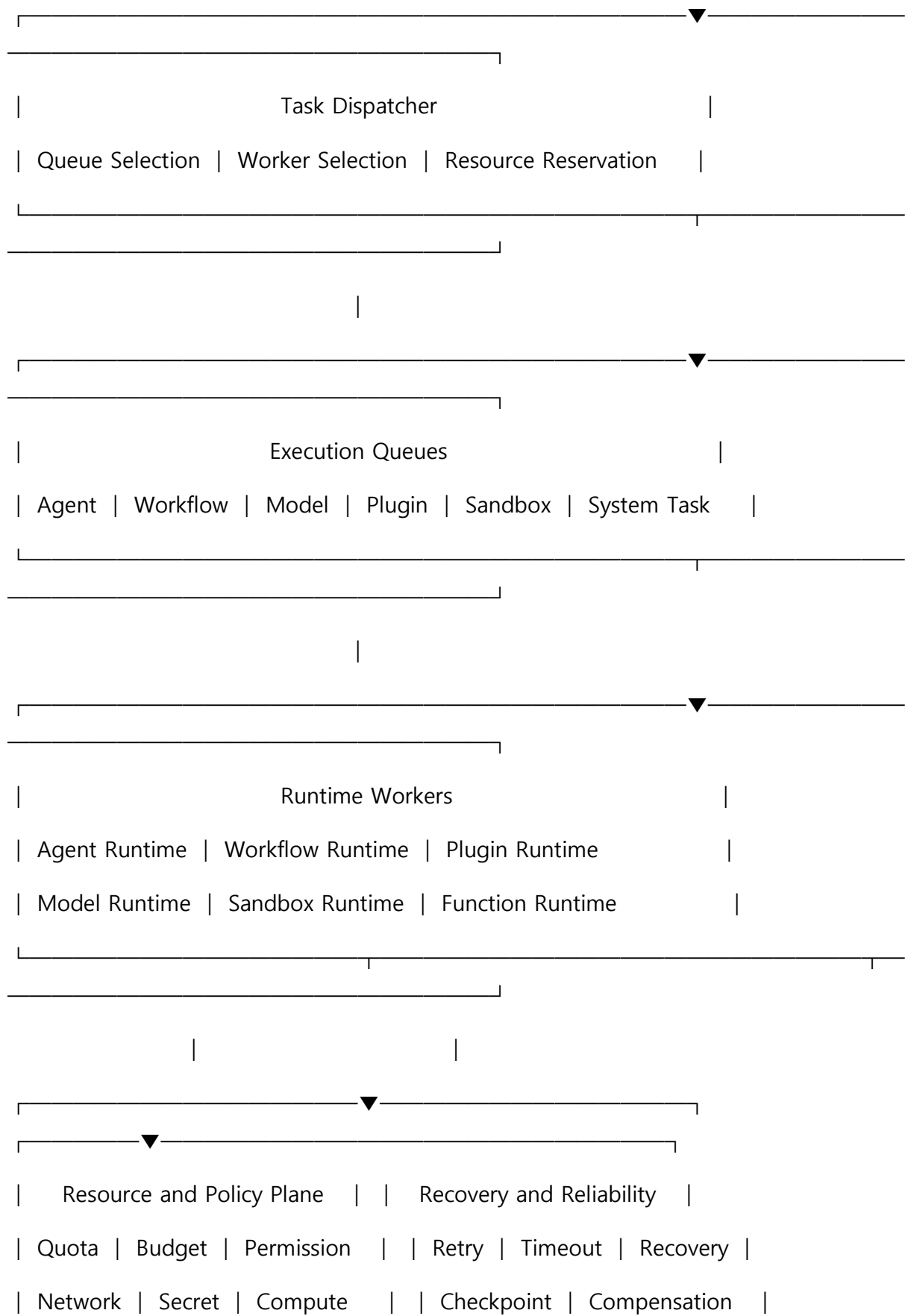
4. Runtime 전체 구조

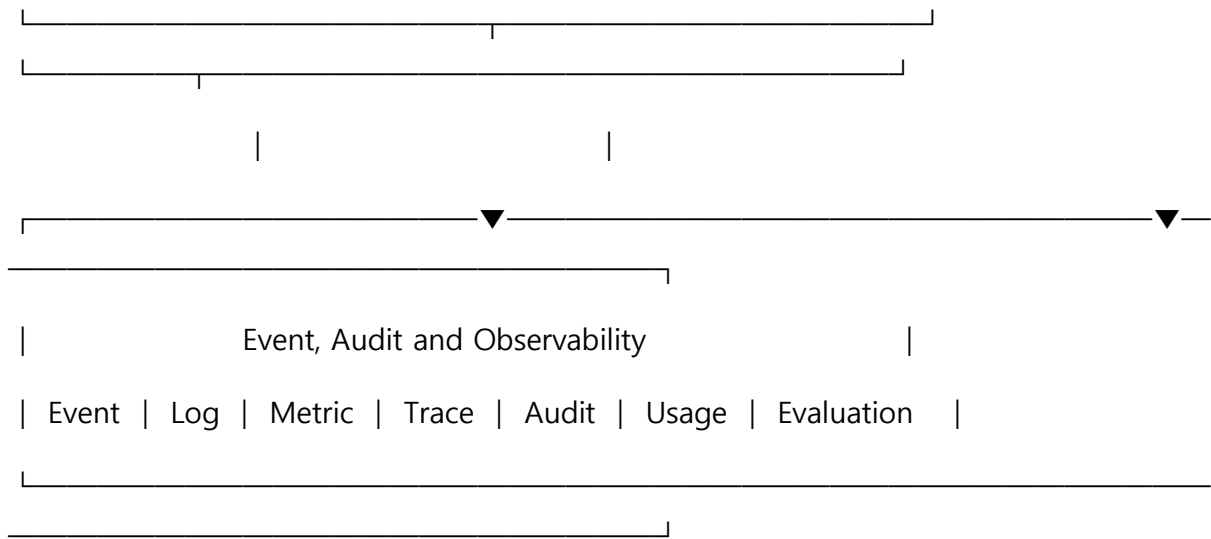
AGEX Runtime은 다음 주요 영역으로 구성한다.

1. Runtime Gateway
2. Execution Manager
3. Execution Scheduler
4. Task Planner
5. Task Dispatcher
6. Execution Queue
7. Runtime Worker
8. Agent Runtime
9. Workflow Runtime
10. Plugin Runtime
11. Model Runtime
12. Sandbox Runtime
13. State Manager
14. Checkpoint Manager
15. Resource Manager
16. Retry and Timeout Manager
17. Recovery Manager
18. Compensation Manager
19. Runtime Policy Engine
20. Runtime Event Manager
21. Runtime Observability
22. Runtime Storage Adapters

5. Runtime 개념 구조







6. Runtime Control Plane과 Data Plane

6.1 Runtime Control Plane

Runtime Control Plane은 실행의 생성, 구성, 배치, 상태 및 정책을 관리한다.

주요 책임은 다음과 같다.

- Execution 생성
- Execution Snapshot 검증
- 실행 우선순위 결정
- Task Graph 생성
- Worker 배치
- 자원 예약
- 상태 전이
- Retry 결정
- Timeout 결정
- 취소 및 중단
- Recovery 조정
- Runtime Configuration 관리

6.2 Runtime Data Plane

Runtime Data Plane은 실제 Task를 수행한다.

주요 책임은 다음과 같다.

- Agent Loop 실행
- Workflow Step 실행
- Model 호출
- Plugin 호출
- Function 실행
- Knowledge 조회
- Memory 조회 및 기록
- Artifact 처리
- 외부 Callback 처리

6.3 분리 원칙

Control Plane과 Data Plane은 다음 원칙으로 분리한다.

- Control Plane은 실제 사용자 코드를 실행하지 않는다.
- Data Plane은 정책이나 권한을 임의로 변경하지 않는다.
- Data Plane은 Control Plane이 제공한 Execution Snapshot 안에서만 동작한다.
- Control Plane 장애 시 진행 중인 Data Plane Task는 안전한 범위에서 계속 실행할 수 있어야 한다.
- Data Plane은 상태 변경을 State Manager를 통해 기록한다.

7. Runtime Gateway

7.1 정의

Runtime Gateway는 Core의 Execution Coordination 영역에서 전달된 실행 요청을 Runtime 내부 실행 형식으로 변환하는 진입점이다.

7.2 입력 조건

Runtime Gateway는 다음 정보가 포함된 요청만 수신한다.

- Execution ID
- Tenant ID
- Principal ID
- Target Resource
- Target Version
- Execution Snapshot
- Input
- Permission Context
- Policy Context
- Budget
- Quota
- Priority
- Deadline
- Runtime Requirements
- Trace Context

7.3 검증 항목

Runtime Gateway는 다음을 재검증한다.

- Execution Snapshot 무결성
- Resource Version 유효성
- Tenant 상태
- 실행 권한
- Policy Decision 유효기간
- Budget
- Quota

- Runtime 호환성
- 필수 Dependency 존재 여부
- 요청 중복 여부
- 실행 취소 여부

7.4 실행 거부 조건

다음 조건에서는 실행을 거부한다.

- Snapshot 변조
- Resource Version 미존재
- 권한 만료
- Policy 변경으로 인한 금지
- Tenant 정지
- Quota 초과
- Budget 초과
- Runtime Version 비호환
- 필수 Plugin 비활성화
- 필수 Model 사용 불가
- 중복 Execution 요청
- Deadline 경과

8. Execution Model

8.1 Execution 정의

Execution은 AGEX Runtime에서 관리되는 하나의 독립 실행 인스턴스이다.

Execution은 다음 중 하나를 대상으로 생성할 수 있다.

- Agent
- Workflow

- Plugin Action
- Function
- Model Task
- System Task
- Sub-Execution

8.2 Execution 공통 속성

모든 Execution은 다음 속성을 가진다.

- Execution ID
- Execution Type
- Tenant ID
- Principal ID
- Target Resource ID
- Target Resource Version
- Parent Execution ID
- Root Execution ID
- Correlation ID
- Causation ID
- Priority
- State
- Input
- Output
- Error
- Execution Snapshot
- Budget
- Usage

- Created At
- Scheduled At
- Started At
- Completed At
- Deadline
- Retry Count
- Checkpoint Reference
- Runtime Worker Reference
- Trace Context

8.3 Root Execution과 Child Execution

최초 요청은 Root Execution으로 생성한다.

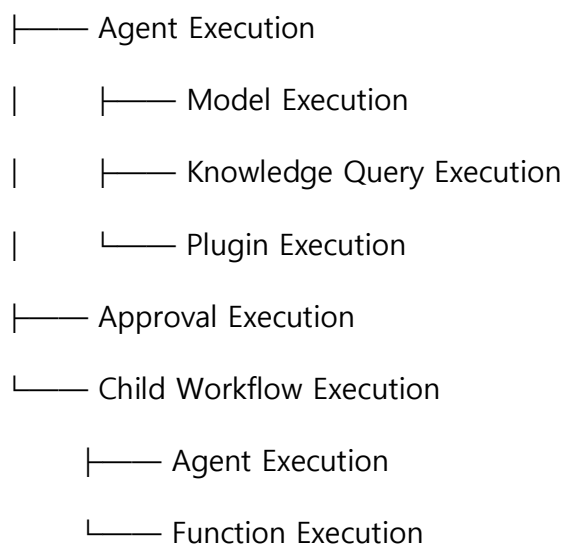
Agent가 Plugin, Model 또는 다른 Agent를 호출하면 Child Execution을 생성할 수 있다.

모든 Child Execution은 Root Execution과 Parent Execution을 참조해야 한다.

8.4 Execution Tree

복합 실행은 Tree 구조로 관리한다.

Root Workflow Execution



8.5 Execution Snapshot

모든 Execution은 시작 시점에 확정된 불변 Snapshot을 사용한다.

Snapshot에는 다음이 포함된다.

- Resource Definition
- Agent Definition
- Workflow Definition
- Plugin Manifest
- Model Policy
- Prompt Version
- Knowledge Scope
- Memory Policy
- Permission Scope
- Policy Decision
- Runtime Configuration
- Environment Configuration
- Cost Limit
- Resource Limit
- Schema Version

9. Execution State Machine

9.1 기본 상태

Execution은 다음 상태를 가진다.

- Created
- Validating
- Queued
- Scheduled

- Assigned
- Starting
- Running
- Waiting
- WaitingForEvent
- WaitingForApproval
- Pausing
- Paused
- Resuming
- Retrying
- Compensating
- Cancelling
- Completed
- Failed
- Cancelled
- TimedOut
- Terminated

9.2 정상 상태 흐름

Created

→ Validating

→ Queued

→ Assigned

→ Starting

→ Running

→ Completed

9.3 대기 상태 흐름

Running

→ Waiting

→ Queued

→ Assigned

→ Running

9.4 일시정지 흐름

Running

→ Pausing

→ Paused

→ Resuming

→ Queued

→ Running

9.5 재시도 흐름

Running

→ Failed Attempt

→ Retrying

→ Scheduled

→ Queued

→ Running

9.6 취소 흐름

Running

→ Cancelling

→ Cancelled

9.7 상태 전이 규칙

모든 상태 전이는 다음을 포함해야 한다.

- 허용된 이전 상태
- 다음 상태
- 전이 주체
- 전이 조건
- 저장할 상태 정보
- 발행할 Event
- Audit 필요 여부
- 실패 시 처리

9.8 Terminal State

다음은 Terminal State이다.

- Completed
- Failed
- Cancelled
- TimedOut
- Terminated

Terminal State에 도달한 Execution은 다시 Running 상태로 전환하지 않는다.

재실행이 필요한 경우 새로운 Execution을 생성한다.

10. Execution Manager

10.1 정의

Execution Manager는 Execution의 전체 Lifecycle을 관리한다.

10.2 주요 기능

- Execution 생성
- 실행 시작

- 일시정지
- 재개
- 취소
- 강제 종료
- 상태 조회
- Child Execution 생성
- 결과 확정
- 오류 확정
- Completion 처리
- Cleanup 요청

10.3 실행 생성

Execution 생성 시 다음을 수행한다.

1. 요청 식별
2. 중복 확인
3. Snapshot 저장
4. 초기 State 생성
5. Budget 예약
6. Quota 예약
7. Audit 기록
8. ExecutionCreated Event 발행
9. Scheduler 등록

10.4 실행 완료

실행 완료 시 다음을 수행한다.

1. Output Schema 검증
2. 결과 저장

3. Usage 확정
4. Budget 정산
5. Quota 해제
6. Memory 저장 정책 적용
7. Artifact 등록
8. 상태를 Completed로 전환
9. Audit 기록
10. ExecutionCompleted Event 발행
11. Parent Execution에 결과 전달

10.5 실행 실패

실행 실패 시 다음을 수행한다.

1. 오류 분류
2. Retry 가능 여부 판단
3. Checkpoint 확인
4. Retry Budget 확인
5. 재시도 또는 최종 실패 결정
6. 상태 저장
7. Audit 기록
8. Failure Event 발행
9. Parent Execution에 오류 전달

11. Execution Scheduler

11.1 정의

Execution Scheduler는 실행 시점, 우선순위, 자원 가용성 및 의존성을 고려하여 Execution을 Queue에 배치한다.

11.2 Scheduling 입력

Scheduler는 다음 정보를 사용한다.

- Priority
- Scheduled Time
- Deadline
- Tenant Quota
- Resource Requirement
- Worker Availability
- Execution Type
- Dependency State
- Retry Backoff
- Cost Policy
- Environment
- Data Locality

11.3 우선순위

우선순위는 다음과 같이 구분할 수 있다.

- Critical
- High
- Normal
- Low
- Background

11.4 우선순위 남용 방지

Tenant 또는 Principal이 모든 Execution을 Critical로 지정할 수 없다.

우선순위 사용 권한과 Tenant별 비율 제한을 적용한다.

11.5 공정성

Scheduler는 특정 Tenant가 Runtime 자원을 독점하지 않도록 Fair Scheduling을 적용해야 한다.

가능한 방식은 다음과 같다.

- Weighted Fair Queue
- Tenant Quota
- Concurrency Limit
- Token Bucket
- Priority Aging

11.6 Deadline Scheduling

Deadline이 있는 Execution은 예상 실행 시간과 Queue 지연을 고려하여 수락 여부를 판단할 수 있어야 한다.

완료 가능성이 없는 요청은 조기에 거부하거나 경고해야 한다.

12. Task Planner

12.1 정의

Task Planner는 Execution Definition을 Runtime이 처리할 수 있는 Task Graph로 변환한다.

12.2 Task 유형

- Agent Task
- Workflow Step Task
- Model Task
- Plugin Task
- Function Task
- Knowledge Task
- Memory Task
- Event Task

- Approval Task
- Wait Task
- Compensation Task
- System Task

12.3 Task 공통 속성

- Task ID
- Execution ID
- Task Type
- State
- Input
- Output
- Dependencies
- Retry Policy
- Timeout Policy
- Resource Requirement
- Permission Scope
- Policy Context
- Assigned Worker
- Attempt
- Created At
- Started At
- Completed At

12.4 Task Graph

Task Graph는 Directed Acyclic Graph를 기본으로 한다.

반복이나 순환은 Graph 자체 순환이 아니라 명시적 Loop Controller를 통해 표현한다.

12.5 병렬 실행

의존성이 없는 Task는 병렬 실행할 수 있다.

병렬 실행 시 다음을 검토한다.

- 공유 상태 충돌
- 비용 증가
- Rate Limit
- Plugin 제한
- Model 동시 호출 제한
- Parent Execution Timeout
- 결과 병합 방식

12.6 동적 Task 생성

Agent가 계획을 생성하거나 Workflow가 조건에 따라 분기하는 경우 동적으로 Task를 추가할 수 있다.

동적 Task도 다음 검증을 거쳐야 한다.

- Schema 검증
- Permission 검증
- Policy 검증
- Budget 검증
- 최대 Task 수 검증
- 최대 Depth 검증

13. Task Dispatcher

13.1 정의

Task Dispatcher는 실행 가능한 Task를 적절한 Queue와 Worker에 전달한다.

13.2 Worker 선택 조건

Worker 선택 시 다음을 고려한다.

- Task Type
- Runtime Capability
- Region
- Environment
- Security Level
- Tenant Isolation Requirement
- Resource Requirement
- Data Locality
- Worker Load
- Runtime Version
- Required Adapter
- Required Sandbox

13.3 Task Lease

Worker는 Task를 영구 소유하지 않고 제한된 시간의 Lease를 획득한다.

Lease가 만료되면 Task는 재배치할 수 있다.

13.4 Heartbeat

장기 실행 Worker는 주기적으로 Heartbeat를 전송해야 한다.

Heartbeat에는 다음이 포함된다.

- Worker ID
- Task ID
- Execution ID
- Current State
- Progress
- Resource Usage

- Last Checkpoint
- Estimated Remaining Policy
- Health Status

13.5 Worker 손실

Heartbeat가 일정 시간 이상 수신되지 않으면 Worker Loss로 판단한다.

Task는 Checkpoint와 Idempotency 정책에 따라 복구 또는 재실행한다.

14. Execution Queue

14.1 Queue 유형

Runtime은 Task 특성에 따라 Queue를 구분할 수 있다.

- Agent Queue
- Workflow Queue
- Model Queue
- Plugin Queue
- Sandbox Queue
- High Priority Queue
- Scheduled Queue
- Retry Queue
- Compensation Queue
- Dead Letter Queue

14.2 Queue Message

Queue Message에는 전체 민감 데이터를 포함하지 않는다.

최소한 다음 Reference만 포함한다.

- Task ID
- Execution ID

- Tenant ID
- Task Type
- Priority
- Snapshot Reference
- Attempt
- Scheduled Time
- Trace Context

14.3 전달 보장

Queue는 최소 1회 전달을 기본으로 한다.

따라서 Task Handler는 Idempotent해야 한다.

14.4 중복 처리

Task Attempt ID와 Idempotency Key를 이용하여 중복 실행을 방지한다.

14.5 Queue Backpressure

Queue 지연이 증가할 경우 다음 정책을 적용할 수 있다.

- 신규 실행 제한
- 낮은 우선순위 지연
- Tenant별 동시 실행 축소
- Worker 자동 확장
- 외부 호출 제한
- 운영 Alert 발생

15. Runtime Worker

15.1 정의

Runtime Worker는 Queue에서 Task를 수신하여 실제 작업을 수행하는 실행 프로세스이다.

15.2 Worker 유형

- Agent Worker
- Workflow Worker
- Model Worker
- Plugin Worker
- Sandbox Worker
- System Worker

15.3 Stateless 원칙

Worker는 가능한 한 Stateless하게 구현한다.

필요한 상태는 State Store와 Checkpoint Store에서 읽는다.

15.4 Worker 시작 절차

Worker는 Task를 시작하기 전에 다음을 수행한다.

1. Lease 획득
2. Task 상태 확인
3. 취소 여부 확인
4. Snapshot 검증
5. Permission Scope 로드
6. Policy Context 로드
7. Secret Reference 확인
8. Resource 예약
9. 상태를 Starting으로 변경
10. TaskStarted Event 발행

15.5 Worker 종료 절차

Worker는 Task 종료 시 다음을 수행한다.

1. Output 수집

2. Output Schema 검증
 3. Usage 기록
 4. Checkpoint 저장
 5. Task 상태 변경
 6. Resource 해제
 7. Lease 해제
 8. Event 발행
 9. Parent Task에 결과 전달
-

16. Agent Runtime

16.1 정의

Agent Runtime은 Agent의 목표 해석, 계획, 도구 사용, 상태 유지 및 결과 생성을 실행하는 전용 Runtime이다.

16.2 Agent 실행 단계

Agent Runtime은 다음 단계를 수행한다.

1. Agent Definition 로드
2. Agent Version 확인
3. Input 검증
4. Context 구성
5. Memory 조회
6. Knowledge 조회
7. Model 선택
8. 계획 생성 또는 기존 계획 로드
9. Action 선택
10. Tool 또는 Plugin 실행

11. 결과 관찰
12. 상태 갱신
13. 완료 여부 판단
14. 결과 검증
15. Memory 저장 정책 적용
16. Output 반환

16.3 Agent Loop

Agent Runtime은 반복 실행 Loop를 지원한다.

Observe

- Understand
- Plan
- Select Action
- Execute
- Validate
- Update State
- Continue or Complete

16.4 Loop 제한

무한 실행을 방지하기 위해 다음 제한을 적용한다.

- 최대 Iteration
- 최대 Model 호출 횟수
- 최대 Plugin 호출 횟수
- 최대 Child Execution 수
- 최대 Token
- 최대 비용
- 최대 실행 시간

- 최대 오류 횟수

16.5 Agent Plan

Agent Plan은 다음 정보를 포함할 수 있다.

- Goal
- Assumptions
- Tasks
- Dependencies
- Required Tools
- Completion Criteria
- Risk
- Estimated Cost
- Estimated Duration

16.6 Plan 검증

생성된 Plan은 실행 전에 다음을 검증한다.

- 허용된 Capability 범위
- 사용 가능한 Plugin
- Permission Scope
- Policy
- 비용
- Task 수
- 실행 Depth
- 외부 영향
- 승인 필요 여부

16.7 Agent 간 호출

Agent가 다른 Agent를 호출할 경우 Child Execution으로 생성한다.

호출 Agent의 권한이 대상 Agent에 자동 상속되지 않는다.

대상 Agent는 자신의 Permission과 Policy Scope를 사용한다.

17. Workflow Runtime

17.1 정의

Workflow Runtime은 Workflow Definition에 따라 Step을 실행하고 상태를 지속적으로 관리한다.

17.2 Workflow Instance

Workflow 실행 시 Workflow Instance를 생성한다.

주요 속성은 다음과 같다.

- Workflow Instance ID
- Workflow ID
- Workflow Version
- Current State
- Current Steps
- Variables
- Completed Steps
- Failed Steps
- Pending Steps
- Child Executions
- Checkpoint
- Started At
- Updated At
- Completed At

17.3 Step 유형

- Start
- End
- Agent
- Model
- Plugin
- Function
- Condition
- Switch
- Parallel
- Join
- Loop
- Wait
- Event Wait
- Approval
- Sub-Workflow
- Compensation
- Error Handler

17.4 Step 상태

- Pending
- Ready
- Queued
- Running
- Waiting
- Completed
- Failed

- Skipped
- Cancelled
- Compensated

17.5 Workflow 변수

Workflow 변수는 Scope를 구분한다.

- Global Scope
- Workflow Scope
- Branch Scope
- Step Scope
- Secret Scope

17.6 상태 지속성

각 중요한 Step 완료 후 Workflow 상태를 저장한다.

긴 Workflow는 모든 Step 이후 또는 정책에 정의된 간격으로 Checkpoint를 생성한다.

17.7 Workflow 재생

Workflow Runtime은 실행 이력을 기준으로 상태를 재구성하거나 Replay할 수 있어야 한다.

단, 외부 Side Effect는 Replay 중 재실행하지 않도록 구분해야 한다.

18. Model Runtime

18.1 정의

Model Runtime은 다양한 Model Provider 호출을 공통 실행 방식으로 처리한다.

18.2 주요 책임

- Model Routing
- Provider 선택
- Credential 중개

- Prompt 구성
- Input 변환
- Context Window 관리
- Token Budget 관리
- Streaming
- Structured Output
- Output Validation
- Retry
- Fallback
- Usage 측정
- 비용 계산

18.3 Model 요청 구조

- Model Task ID
- Capability
- Preferred Model
- Allowed Models
- Input
- Prompt Reference
- Context
- Output Schema
- Temperature
- Token Limit
- Latency Limit
- Cost Limit
- Data Classification

- Region Constraint
- Fallback Policy

18.4 Model Routing

Model Router는 다음 조건으로 Model을 선택한다.

- Required Capability
- Input Modality
- Output Modality
- Context Length
- Quality Requirement
- Latency
- Cost
- Availability
- Data Residency
- Security Classification
- Tenant Policy
- Provider Health

18.5 Structured Output

시스템 상태에 반영되는 Model 출력은 Schema 기반으로 검증해야 한다.

검증 실패 시 다음 중 하나를 수행한다.

- 동일 Model 재요청
- Repair Prompt 사용
- 대체 Model 호출
- Human Review 요청
- Task 실패

18.6 Model Fallback

Fallback은 사전에 정의된 정책에 따라 수행한다.

Model이 실패했다고 임의의 Provider로 데이터를 전송해서는 안 된다.

19. Plugin Runtime

19.1 정의

Plugin Runtime은 Plugin Action을 격리된 방식으로 실행한다.

19.2 Plugin 실행 절차

1. Plugin Installation 확인
2. Plugin Version 확인
3. Action 확인
4. Permission 확인
5. Policy 확인
6. Input Schema 검증
7. Credential 요청
8. Network Policy 적용
9. Runtime 격리
10. Plugin 호출
11. Response 검증
12. Output Schema 검증
13. Usage 기록
14. Credential 폐기 또는 반환
15. 결과 저장

19.3 Credential 보호

Plugin은 Credential 원문을 가능한 한 직접 받지 않는다.

Credential Broker가 다음 방식 중 하나로 제공한다.

- 단기 Token
- 서명된 Request
- Proxy 호출
- 임시 Session
- 제한된 Credential Reference

19.4 Side Effect 분류

Plugin Action은 다음으로 분류한다.

- Read Only
- Reversible Write
- Irreversible Write
- Privileged Action

분류에 따라 승인, Retry 및 Compensation 정책을 다르게 적용한다.

19.5 Plugin Retry

Irreversible Write는 Idempotency가 보장되지 않으면 자동 재시도하지 않는다.

20. Sandbox Runtime

20.1 정의

Sandbox Runtime은 외부 코드, 사용자 코드 또는 신뢰 수준이 낮은 실행을 격리하는 환경이다.

20.2 격리 대상

- 사용자 제출 코드
- 외부 Extension
- Marketplace Package
- 동적 Script
- 파일 변환 코드

- 검증되지 않은 Plugin
- 모델 생성 코드

20.3 격리 방식

- Process Isolation
- Container Isolation
- MicroVM Isolation
- File System Isolation
- Network Isolation
- Syscall Restriction
- CPU Limit
- Memory Limit
- Disk Limit
- Time Limit

20.4 기본 제한

Sandbox는 기본적으로 다음을 금지한다.

- 외부 Network 접근
- Host File System 접근
- Core Database 접근
- Secret Store 접근
- 다른 Tenant Runtime 접근
- Privileged System Call
- 지속 프로세스 생성

필요한 접근은 명시적 Permission으로만 허용한다.

20.5 Sandbox Output

Sandbox 결과는 다음 검증을 거친다.

- Output Size
 - File Type
 - Malware Scan
 - Schema Validation
 - Sensitive Data Detection
 - Policy Validation
-

21. State Manager

21.1 정의

State Manager는 Execution, Task, Workflow 및 Agent의 상태를 영속적으로 관리한다.

21.2 상태 저장 원칙

- 모든 상태 전이는 원자적으로 기록한다.
- 상태는 State Machine을 통해서만 변경한다.
- 상태 변경에는 Revision을 부여한다.
- 중복 상태 변경을 허용하지 않는다.
- 상태 변경 후 Event를 발행한다.
- 상태 변경 주체를 기록한다.

21.3 Execution State Record

- Execution ID
- Current State
- Previous State
- Revision
- Updated At
- Updated By
- Reason

- Worker ID
- Attempt
- Checkpoint Reference
- Error Reference

21.4 상태 경쟁

다수 Worker가 동일 Task를 처리하지 않도록 Compare-And-Swap, Lock 또는 Lease를 사용한다.

21.5 상태 조회

운영자는 Root Execution부터 모든 Child Execution과 Task 상태를 조회할 수 있어야 한다.

22. Checkpoint Manager

22.1 정의

Checkpoint Manager는 실행 복구에 필요한 중간 상태를 저장하고 관리한다.

22.2 Checkpoint 생성 시점

- Workflow Step 완료
- Agent Iteration 완료
- 외부 호출 전
- 외부 호출 후
- 승인 대기 전
- Event 대기 전
- 장기 Task 주기
- Worker 종료 전
- Pause 처리 전

22.3 Checkpoint 내용

- Execution ID

- Task ID
- State
- Variables
- Intermediate Output
- Completed Actions
- Pending Actions
- External References
- Retry Count
- Model Context Reference
- Memory Delta
- Artifact References
- Created At
- Checksum

22.4 Checkpoint 불변성

저장된 Checkpoint는 수정하지 않는다.

새로운 상태는 새로운 Checkpoint로 저장한다.

22.5 Checkpoint 보존

보존 기간은 Execution 유형, Tenant 정책 및 감사 요구사항에 따라 설정한다.

23. Retry Manager

23.1 정의

Retry Manager는 실패 유형에 따라 안전한 재시도를 수행한다.

23.2 Retry 판단 요소

- Error Category
- Retryable 여부

- Task Side Effect
- Idempotency 지원
- Retry Count
- Retry Budget
- Deadline
- Remaining Cost Budget
- External Provider 상태
- Parent Execution 상태

23.3 Retry Policy

Retry Policy는 다음을 포함한다.

- Maximum Attempts
- Initial Delay
- Backoff Strategy
- Maximum Delay
- Jitter
- Retryable Error Codes
- Non-Retryable Error Codes
- Fallback Action
- Exhausted Action

23.4 Backoff 방식

- Fixed
- Linear
- Exponential
- Exponential with Jitter

23.5 Retry Budget

개별 Task 제한 외에 Root Execution 전체의 Retry Budget을 설정한다.

하위 Task가 무제한으로 재시도 비용을 소모하지 못하게 한다.

24. Timeout Manager

24.1 Timeout 유형

- Queue Timeout
- Start Timeout
- Execution Timeout
- Task Timeout
- Model Timeout
- Plugin Timeout
- Idle Timeout
- Approval Timeout
- Event Wait Timeout
- Workflow Deadline
- Root Execution Deadline

24.2 Timeout 처리

Timeout 발생 시 다음 중 하나를 수행한다.

- Task 실패
- 재시도
- Fallback
- Compensation
- Partial Result 반환
- Human Review 요청
- Execution 종료

24.3 계층적 Timeout

Child Task의 Timeout은 Parent Execution의 남은 Deadline을 초과할 수 없다.

24.4 외부 호출 Timeout

모든 외부 Model, Plugin 및 API 호출에는 명시적 Timeout을 설정한다.

무제한 대기는 허용하지 않는다.

25. Recovery Manager

25.1 정의

Recovery Manager는 Worker 장애, Runtime 재시작, Queue 장애 및 인프라 장애 후 실행을 복구한다.

25.2 복구 대상

- Running Task
- Waiting Execution
- Retrying Task
- Paused Execution
- Approval Pending
- Event Waiting
- Compensation Running
- Lease Expired Task

25.3 복구 절차

1. 비정상 상태 탐지
2. Worker Health 확인
3. Lease 만료 확인
4. 마지막 Checkpoint 조회
5. Side Effect 확인

6. Idempotency 확인
7. 복구 가능 여부 결정
8. 새 Worker 할당
9. 상태 전이
10. Recovery Event 발행
11. 실행 재개

25.4 Side Effect 확인

외부 Write 이후 응답을 받지 못한 경우 즉시 재시도하지 않는다.

다음 방식으로 상태를 확인한다.

- Idempotency Key 조회
- 외부 상태 조회
- Callback 기록 조회
- 운영자 확인
- Compensation 수행

25.5 복구 불가능 상태

복구에 필요한 Checkpoint가 없거나 외부 Side Effect 상태를 확인할 수 없는 경우 Manual Recovery 상태로 전환할 수 있다.

26. Compensation Manager

26.1 정의

Compensation Manager는 복합 실행의 일부가 완료된 후 전체 실행을 되돌려야 할 때 보상 작업을 수행한다.

26.2 Compensation 대상

- 외부 데이터 생성
- 외부 상태 변경
- 자원 예약

- 임시 Credential 생성
- Artifact 생성
- Child Execution 완료
- 비용 예약

26.3 Compensation 정의

Side Effect가 있는 Task는 가능한 경우 다음을 선언한다.

- Compensation Action
- Compensation Input Mapping
- Compensation Permission
- Compensation Timeout
- Compensation Retry
- Compensation Failure Policy

26.4 실행 순서

기본적으로 완료된 작업의 역순으로 보상한다.

26.5 보상 실패

Compensation 실패 시 자동으로 성공 처리하지 않는다.

실패 상태와 미복구 자원을 명시하고 운영자 개입을 요청한다.

27. Resource Manager

27.1 정의

Resource Manager는 Execution과 Task가 사용하는 Compute, Memory, Token, Storage 및 동시 실행 자원을 통제한다.

27.2 관리 대상

- CPU
- Memory

- GPU
- Local Disk
- Network
- Token
- Model Concurrency
- Plugin Concurrency
- Sandbox Count
- Worker Slot
- Queue Slot
- Execution Time

27.3 Resource Request

Task는 실행 전에 필요한 자원을 선언한다.

- Minimum Resource
- Preferred Resource
- Maximum Resource
- Runtime Class
- Accelerator Requirement
- Region
- Data Locality
- Security Level

27.4 Resource Reservation

Scheduler는 실행 전에 자원을 예약할 수 있다.

예약 실패 시 Queue 대기, 대체 Runtime 선택 또는 실행 거부를 수행한다.

27.5 자원 초과

Task가 할당량을 초과하면 다음 중 하나를 적용한다.

- Throttling
 - Pause
 - Graceful Termination
 - Forced Termination
 - Scale Up
 - Spill to External Storage
-

28. Runtime Policy Enforcement

28.1 정의

Runtime Policy Enforcement는 실행 직전과 실행 중 정책을 지속적으로 적용한다.

28.2 실행 전 정책

- Tenant 상태
- Principal 권한
- Resource 권한
- Model 사용 권한
- Plugin 사용 권한
- Knowledge 접근 권한
- Memory 접근 권한
- Data Classification
- Region
- Network
- Cost
- Time
- Approval

28.3 실행 중 정책

- 누적 비용
- Token 사용량
- Plugin 호출 횟수
- Model 호출 횟수
- Runtime 시간
- Child Execution 수
- Agent Iteration 수
- 데이터 이동
- Secret 접근
- 외부 Endpoint 접근

28.4 정책 변경

실행 중 정책이 변경될 경우 Resource 유형과 위험 수준에 따라 다음 중 하나를 적용한다.

- 기존 Snapshot 유지
- 다음 Task부터 신규 정책 적용
- 실행 일시정지
- 실행 중단
- 재승인 요구

28.5 Kill Policy

중대한 보안 위험이 탐지되면 Snapshot과 관계없이 실행을 즉시 중단할 수 있어야 한다.

29. Cancellation and Termination

29.1 취소 정의

Cancellation은 정상적인 종료 절차를 수행하며 실행을 중지하는 요청이다.

29.2 강제 종료 정의

Termination은 보안, 장애 또는 운영상 이유로 실행을 즉시 중단하는 조치이다.

29.3 취소 가능 지점

Worker는 다음 Safe Point에서 취소를 확인한다.

- Task 시작 전
- Agent Iteration 전후
- Workflow Step 전후
- 외부 호출 전
- Checkpoint 저장 후
- Wait 상태 진입 전

29.4 취소 처리

취소 시 다음을 수행한다.

- 신규 Child Task 생성 중지
- 진행 중 외부 호출 중단 시도
- Checkpoint 저장
- 예약 자원 해제
- 필요 시 Compensation 실행
- 상태를 Cancelled로 전환
- Audit 및 Event 기록

29.5 강제 종료

강제 종료는 Checkpoint나 Compensation을 보장하지 않을 수 있다.

따라서 이유, 주체 및 영향 범위를 반드시 Audit에 기록한다.

30. Approval and External Wait

30.1 Approval Wait

승인이 필요한 Task는 WaitingForApproval 상태로 전환한다.

30.2 Approval Token

승인 요청은 다음 정보를 가진다.

- Approval ID
- Execution ID
- Task ID
- Requested Action
- Risk Level
- Requester
- Required Approver
- Expiration
- Input Summary
- Expected Side Effect
- Cost Estimate

30.3 승인 후 재검증

승인 후 다음을 다시 검증한다.

- 승인 유효성
- 정책
- 권한
- Resource 상태
- Budget
- Deadline
- 외부 상태

30.4 Event Wait

외부 Event를 기다리는 Execution은 다음 정보를 저장한다.

- Event Type

- Correlation Key
- Filter
- Expiration
- Resume Point
- Duplicate Handling Policy

30.5 Callback 보안

외부 Callback은 서명, Token, Nonce 또는 Mutual Authentication으로 검증한다.

31. Runtime Storage

31.1 저장 데이터 유형

- Execution Metadata
- Execution Snapshot
- Task State
- Workflow State
- Agent State
- Checkpoint
- Lease
- Lock
- Output
- Error
- Usage
- Event
- Artifact Reference

31.2 저장소 역할 분리

- Transactional Store: 상태와 Metadata

- Queue: 실행 전달
- Object Store: 대용량 Input, Output, Artifact
- Cache: 임시 조회
- Event Store: 실행 Event
- Search Store: 운영 조회
- Metrics Store: 성능과 사용량

31.3 Source of Truth

Execution State의 공식 Source of Truth는 Transactional State Store이다.

Queue나 Cache는 Source of Truth로 사용하지 않는다.

31.4 대용량 데이터

대용량 Input과 Output은 Queue Message나 상태 Table에 직접 저장하지 않고 Object Reference를 사용한다.

32. Runtime Event

32.1 주요 Event

- ExecutionCreated
- ExecutionQueued
- ExecutionAssigned
- ExecutionStarted
- ExecutionPaused
- ExecutionResumed
- ExecutionWaiting
- ExecutionRetrying
- ExecutionCompleted
- ExecutionFailed

- ExecutionCancelled
- ExecutionTimedOut
- ExecutionTerminated
- TaskStarted
- TaskCompleted
- TaskFailed
- CheckpointCreated
- WorkerLost
- RecoveryStarted
- RecoveryCompleted
- CompensationStarted
- CompensationFailed
- BudgetThresholdReached
- PolicyViolationDetected

32.2 Event 순서

Execution 단위 Sequence Number를 사용하여 상태 Event의 순서를 확인할 수 있어야 한다.

32.3 Event 불변성

발행된 Runtime Event는 수정하지 않는다.

정정이 필요한 경우 새로운 Correction Event를 발행한다.

33. Runtime Observability

33.1 Log

모든 Runtime Log는 구조화된 형식으로 생성한다.

필수 필드는 다음과 같다.

- Timestamp
- Runtime Service
- Worker ID
- Tenant ID
- Execution ID
- Task ID
- Root Execution ID
- Parent Execution ID
- Attempt
- State
- Trace ID
- Span ID
- Message
- Error Code
- Resource Usage

33.2 Metric

다음 Metric을 수집한다.

- Active Execution
- Queued Execution
- Execution Throughput
- Execution Duration
- Task Duration
- Queue Wait Time
- Worker Utilization
- Worker Failure

- Retry Count
- Timeout Count
- Cancellation Count
- Recovery Count
- Checkpoint Duration
- Model Latency
- Plugin Latency
- Token Usage
- Cost
- Tenant Concurrency

33.3 Trace

하나의 Root Execution에서 발생한 모든 Child Execution, Model 호출, Plugin 호출 및 Storage 접근을 단일 Trace로 연결한다.

33.4 실행 Timeline

운영자는 실행 Timeline을 통해 다음을 확인할 수 있어야 한다.

- 상태 전이
- Task 시작과 종료
- Model 호출
- Plugin 호출
- 대기 시간
- Retry
- Timeout
- Approval
- Recovery
- 비용 누적

33.5 Progress

장기 실행은 가능한 경우 진행률 또는 현재 단계를 제공해야 한다.

진행률 계산이 불가능하면 현재 상태와 완료된 Task 수를 제공한다.

34. Runtime Audit

34.1 감사 대상

- Execution 시작
- Execution 취소
- 강제 종료
- 재실행
- Manual Recovery
- 정책 예외
- 승인
- Secret 사용
- Plugin Write
- 외부 Side Effect
- Runtime Configuration 변경
- Worker 격리 해제
- Checkpoint 삭제

34.2 Audit와 Log 구분

Log는 운영을 위한 기술 기록이다.

Audit는 책임 추적과 통제를 위한 불변 기록이다.

Audit 대상 행위를 일반 Log로만 남겨서는 안 된다.

35. Runtime Security

35.1 실행 Identity

모든 Execution은 독립적인 Execution Identity 또는 제한된 Runtime Credential을 가져야 한다.

35.2 권한 상속 제한

Parent Execution의 모든 권한을 Child Execution에 자동 상속하지 않는다.

Child Execution에 필요한 최소 Scope만 전달한다.

35.3 Secret 수명

Secret은 Task 실행에 필요한 최소 시간 동안만 사용할 수 있어야 한다.

가능한 경우 단기 Credential을 사용한다.

35.4 Network Policy

Runtime 유형별로 허용 가능한 Network Endpoint를 제한한다.

35.5 Tenant 격리

Tenant별 실행은 최소한 논리적으로 격리한다.

보안 수준에 따라 별도 Worker Pool, Namespace, Container 또는 Cluster를 사용할 수 있다.

35.6 Prompt Injection 대응

Model이 Plugin이나 Tool을 호출하는 경우 Model 출력만으로 실행하지 않는다.

Tool Call은 다음을 다시 검증한다.

- Agent Capability
- Plugin Permission
- Input Schema
- Policy
- Data Scope
- Side Effect
- Approval Requirement

35.7 Runtime Image

Runtime Image는 서명되고 검증된 Image만 사용한다.

취약점 기준을 초과한 Image는 신규 실행에 사용할 수 없다.

36. Multi-Tenant Runtime

36.1 Tenant Context

모든 Execution, Task, Queue Message, Checkpoint 및 Artifact는 Tenant ID를 포함해야 한다.

36.2 Tenant별 제한

- 동시 Execution 수
- Worker Slot
- Model 호출량
- Plugin 호출량
- Token
- Compute
- Storage
- Queue Depth
- Execution Duration

36.3 Noisy Neighbor 방지

다음 수단을 적용한다.

- Tenant별 Queue
- Weighted Scheduling
- Concurrency Limit
- Rate Limit
- Resource Reservation
- Worker Pool Isolation

- Cost Budget

36.4 Tenant 정지

Tenant가 정지되면 다음 정책을 적용할 수 있다.

- 신규 실행 거부
 - Queued 실행 취소
 - Running 실행 일시정지
 - 보안 위험 시 즉시 종료
 - 승인 대기 실행 만료
-

37. Runtime 비용 관리

37.1 비용 항목

- Model Input Token
- Model Output Token
- Model Request
- Plugin Request
- Compute Time
- GPU Time
- Memory Usage
- Storage
- Network
- Sandbox Execution
- External Service Cost

37.2 비용 집계 단위

- Task
- Execution

- Root Execution
- Agent
- Workflow
- Plugin
- Model
- Tenant
- Application Layer
- Time Period

37.3 실시간 비용 제한

Execution은 누적 비용을 주기적으로 확인한다.

한도 접근 시 다음을 수행할 수 있다.

- 경고
- 저비용 Model로 전환
- 병렬 실행 축소
- 추가 Task 생성 금지
- 승인 요청
- 실행 중단

37.4 비용 예측

Task Planner는 가능한 경우 실행 전 예상 비용 범위를 산정한다.

예상치와 실제 비용의 차이는 기록하고 Routing 및 계획 개선에 활용한다.

38. Runtime 확장 구조

38.1 확장 대상

- Runtime Worker Type
- Task Executor

- Model Adapter
- Plugin Executor
- Sandbox Provider
- Queue Provider
- State Store Adapter
- Checkpoint Store Adapter
- Resource Provider
- Scheduler Strategy
- Retry Strategy

38.2 Runtime Capability

각 Worker는 Capability를 선언한다.

- Supported Task Types
- Runtime Version
- Security Level
- Region
- Resource Class
- Supported Adapters
- Maximum Task Size
- Streaming Support
- Sandbox Support

38.3 호환성

Task는 요구 Runtime Version과 Capability를 선언한다.

호환되지 않는 Worker에는 Task를 배치하지 않는다.

38.4 Extension 제한

Runtime Extension은 다음을 직접 수행할 수 없다.

- State Store 직접 수정
 - Tenant Scope 변경
 - Policy 결과 변경
 - Audit 비활성화
 - Secret 영구 저장
 - Core Queue 임의 접근
-

39. Runtime 배포 구조

39.1 주요 배포 단위

- Runtime Gateway
- Execution Manager
- Scheduler
- Task Dispatcher
- Queue Consumers
- Agent Workers
- Workflow Workers
- Model Workers
- Plugin Workers
- Sandbox Workers
- State Manager
- Recovery Manager

39.2 Worker Pool 분리

Worker Pool은 다음 기준으로 분리할 수 있다.

- Task Type
- Security Level

- Tenant Tier
- Region
- Resource Type
- Runtime Version
- External Network Access
- GPU Requirement

39.3 Autoscaling

다음 Metric을 기반으로 Worker를 확장할 수 있다.

- Queue Depth
- Queue Wait Time
- Active Task Count
- CPU
- Memory
- GPU
- Model Request Rate
- Tenant Demand

39.4 배포 중 실행

Worker 배포 시 진행 중 Task를 즉시 종료하지 않는다.

다음 절차를 적용한다.

1. 신규 Task 수신 중단
 2. 진행 Task 완료 또는 Checkpoint
 3. Lease 반환
 4. Worker 종료
 5. 신규 Version Worker 활성화
-

40. Runtime 장애 시나리오

40.1 Worker 장애

- Lease 만료
- Task 재배포
- Checkpoint 복구
- 중복 Side Effect 확인

40.2 Queue 장애

- 신규 Task 배치 일시 중단
- Outbox 유지
- Queue 복구 후 재전송
- 상태 Source of Truth 기준 복구

40.3 State Store 장애

- 신규 상태 변경 중단
- Worker Safe Stop
- 진행 중 Side Effect 최소화
- 저장소 복구 후 상태 조정

40.4 Model Provider 장애

- Circuit Breaker
- Retry
- 허용된 Fallback
- Queue 대기
- Partial Result
- Task 실패

40.5 Plugin Provider 장애

- Timeout

- Retry
- 상태 확인
- Compensation
- Manual Recovery

40.6 Network 분할

- Lease와 Heartbeat 기준 Worker 상태 판단
 - Split-Brain 방지
 - 중복 Task 실행 방지
 - 복구 후 상태 Reconciliation
-

41. Runtime 테스트 기준

41.1 Unit Test

- State Transition
- Retry Decision
- Timeout Decision
- Scheduling Rule
- Resource Limit
- Cost Calculation

41.2 Contract Test

- Task Contract
- Queue Message
- Worker Capability
- State Store
- Checkpoint
- Runtime Event

41.3 Integration Test

- Scheduler와 Queue
- Worker와 State Store
- Plugin Runtime
- Model Runtime
- Sandbox Runtime
- Recovery Manager

41.4 Failure Test

- Worker Kill
- Queue 중단
- Database 지연
- Network 단절
- Model Timeout
- Plugin 오류
- Disk Full
- Resource Exhaustion

41.5 Recovery Test

- Checkpoint 복구
- Lease 만료 복구
- Runtime 재시작
- Duplicate Delivery
- Partial Side Effect
- Compensation

41.6 Tenant Isolation Test

- Queue 격리

- State 격리
- Secret 격리
- Worker 격리
- Resource Quota 격리

41.7 Load Test

- 대량 Execution 생성
- 동시 Agent 실행
- 장기 Workflow
- 대량 Model 호출
- Queue Backpressure
- Autoscaling

41.8 Soak Test

장시간 지속 부하에서 다음을 검증한다.

- Memory Leak
- Lease 누락
- Lock 누적
- Queue 지연
- Checkpoint 증가
- Worker 불균형
- 비용 집계 오류

42. Runtime 성능 기준

42.1 Control Path

Execution 생성과 상태 조회는 실제 AI 작업과 분리하여 빠르게 응답해야 한다.

42.2 Queue Latency

Queue 대기 시간은 Task 유형과 우선순위별로 측정하고 목표치를 정의해야 한다.

42.3 State Update

상태 저장은 실행 안정성을 보장하면서도 Worker 처리량을 과도하게 제한하지 않아야 한다.

42.4 Checkpoint 비용

Checkpoint 크기와 생성 빈도를 제한한다.

대용량 상태는 Object Store Reference로 분리한다.

42.5 Streaming

Model 또는 Agent 결과 Streaming을 지원하더라도 최종 결과와 상태 확정은 별도 절차로 처리한다.

43. Runtime 금지사항

다음 구조는 허용하지 않는다.

- Execution Snapshot 없는 실행
- Tenant ID 없는 Task
- Worker Local Memory만을 이용한 장기 상태 관리
- 무제한 Agent Loop
- 무제한 Retry
- 무제한 실행 시간
- State Machine을 우회한 상태 변경
- Queue를 Source of Truth로 사용하는 구조
- Idempotency 없는 Write Task 재시도
- Plugin Credential을 Agent에게 직접 노출
- Model 출력만으로 Privileged Action 실행
- Checkpoint 없는 장기 Workflow
- Parent Deadline을 초과하는 Child Task

- Runtime Worker에서 Policy 변경
- Core Database에 Worker가 직접 임의 수정
- Tenant 간 Worker Context 재사용
- Sandbox 없이 외부 코드 실행
- Audit 없는 강제 종료
- 비용 측정 없는 Model 호출
- 검증되지 않은 Runtime Image 사용
- Worker 장애 시 Task 성공으로 간주
- 외부 Side Effect 확인 없이 자동 재실행

44. Runtime Architecture 검증 기준

Runtime Component는 다음 기준을 충족해야 한다.

1. 명확한 실행 책임을 가지는가.
2. Execution State가 정의되어 있는가.
3. Task Contract가 존재하는가.
4. Tenant Context가 포함되는가.
5. Snapshot을 기준으로 실행하는가.
6. Permission과 Policy가 적용되는가.
7. 자원 제한이 정의되는가.
8. Timeout이 정의되는가.
9. Retry가 제한되는가.
10. Idempotency가 검토되었는가.
11. Checkpoint가 필요한지 정의되었는가.
12. 취소와 종료가 가능한가.
13. 실패가 격리되는가.

- 14. 복구 절차가 존재하는가.
- 15. Side Effect와 Compensation이 정의되는가.
- 16. 비용을 측정할 수 있는가.
- 17. Log, Metric, Trace 및 Audit를 제공하는가.
- 18. 특정 Provider와 분리되어 있는가.
- 19. Worker를 수평 확장할 수 있는가.
- 20. Runtime 전체 안정성을 훼손하지 않는가.

45. Runtime Architecture의 최종 정의

AGEX Runtime은 검증된 Agent, Workflow, Plugin, Model 및 Task를 실제로 실행하고, 실행의 전체 Lifecycle과 상태, 자원, 비용, 권한 및 실패를 통제하는 공통 실행 환경이다.

모든 Execution은 불변의 Execution Snapshot을 기준으로 생성되며, Tenant, Identity, Permission, Policy, Budget, Quota 및 Runtime Compatibility 검증을 거쳐 실행된다.

Execution은 명시적인 State Machine으로 관리하고, 장기 실행은 지속적인 Checkpoint를 통해 복구 가능해야 한다.

Task는 Queue와 Worker를 통해 처리하며, 최소 1회 전달을 전제로 Idempotency를 보장해야 한다.

Agent Runtime은 목표 해석과 반복 실행을 담당하고, Workflow Runtime은 선언된 Step과 상태 전이를 관리한다.

Model Runtime은 다양한 Model Provider를 추상화하고, Plugin Runtime은 외부 기능을 권한과 네트워크 정책 안에서 실행한다.

외부 코드와 신뢰 수준이 낮은 작업은 Sandbox Runtime에서 격리한다.

모든 실행은 중단, 취소, 재시도, 복구 및 감사가 가능해야 하며, 무제한 시간, 무제한 비용 또는 무제한 자원을 실행을 허용하지 않는다.

AGEX Runtime Architecture는 다음 문장으로 최종 정의한다.

AGEX Runtime은 모든 AI 실행을 지속 가능하고 복구 가능한 상태로 운영하며, 자율적 행동을 정책, 자원, 비용 및 보안 경계 안에서 통제하는 AI Operating System의 실행 기반이다.

본 문서는 AGEX Runtime의 구조, 실행 모델, 상태 관리, 장애 대응, 보안 및 운영 원칙을 정의하는 공식 Runtime Architecture 문서로 사용한다.